



SUPPORT HUB

Project Documentation

A modern Android-based enterprise helpdesk and ticket management application



CONTENTS

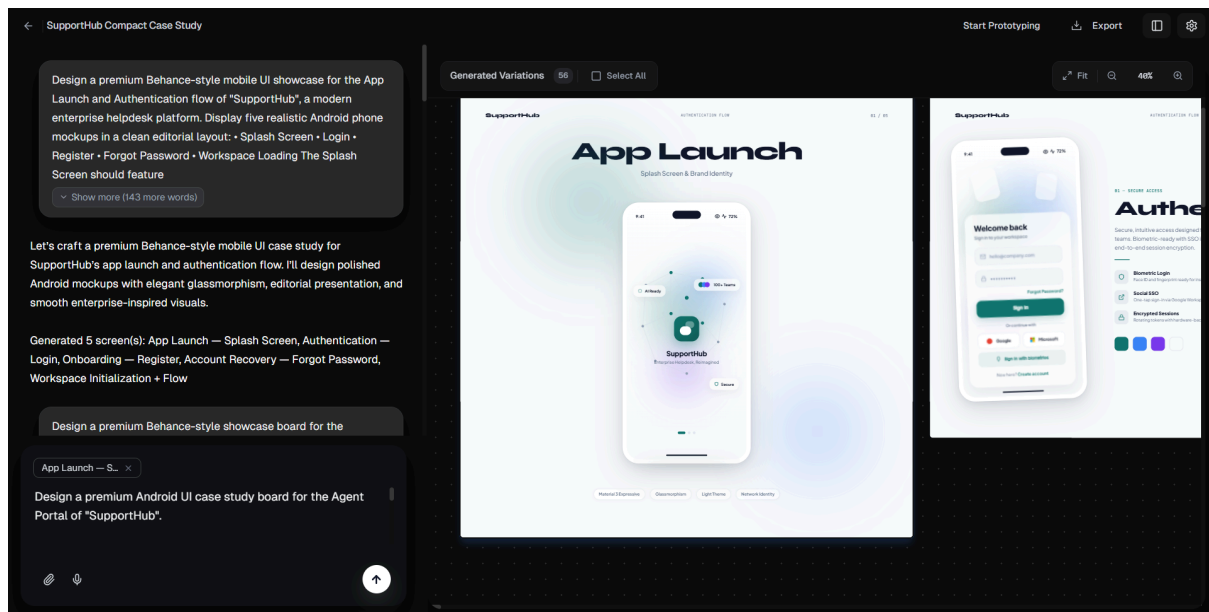
- 01** Project at a Glance
 - 02** Problem, Users & Product Value
 - 03** Customer Persona
 - 04** Architecture & Technology
 - 05** Core Support Workflow
 - 06** Feature Map
 - 07** Build: Design Mode
 - 08** Build: Code Mode
 - 09** Build Prompt Highlights
 - 10** CORA: Planning & Implementation
 - 11** CORA Task Archive
 - 12** Implementation Milestones
 - 13** Backend, Data & Integrations
 - 14** Authentication & User Management
 - 15** Responsive UI & Polish
-

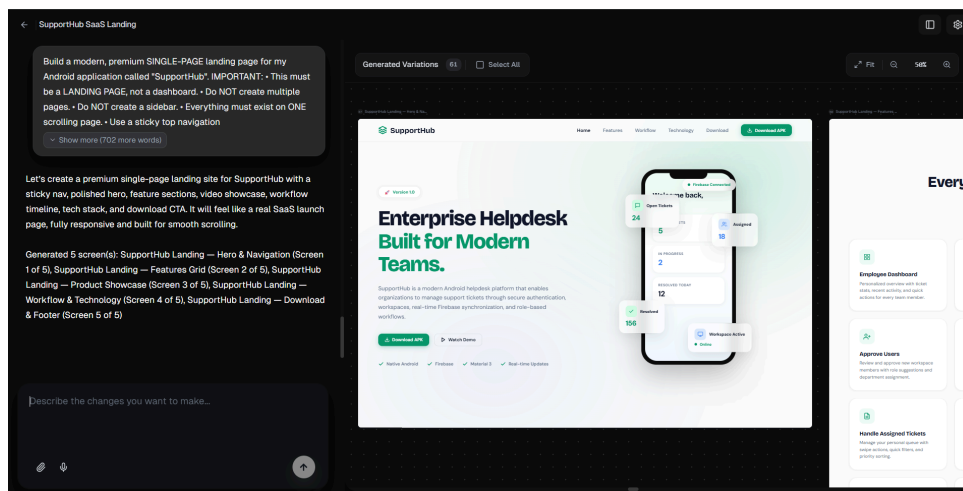
01. PROJECT AT A GLANCE

SupportHub is a modern Android-based enterprise helpdesk and ticket management application designed to streamline organizational support operations.

It provides dedicated experiences for **Employees, Support Agents, and Administrators**, allowing employees to raise and track tickets, agents to manage assigned issues, and administrators to approve users, assign roles, and manage ticket allocation.

The application uses **Kotlin, Jetpack Compose, MVVM, Material 3, and Firebase** with real-time Firestore synchronization.





03. CUSTOMER PERSONA

Primary Persona: Organization Administrator / Support Manager

Goal: Manage organizational support operations from one centralized platform.

Needs:

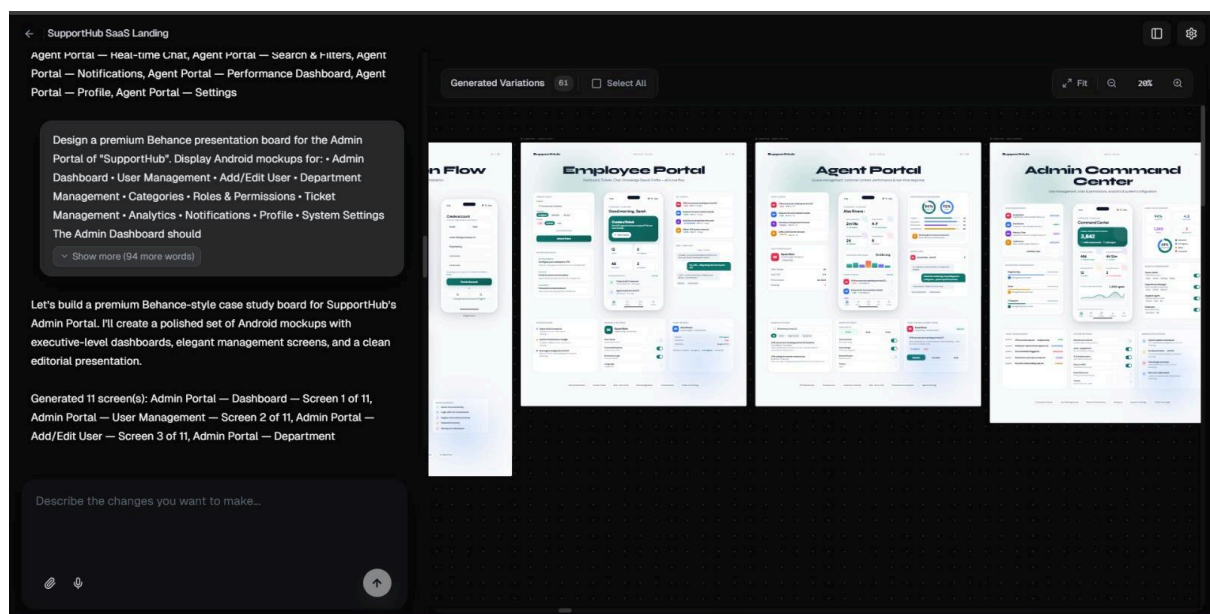
- User and role management
- Ticket assignment
- Support workflow visibility
- Ticket status monitoring
- Employee and agent management

Pain Points:

- Manual ticket handling
- Lack of visibility into ticket progress
- Difficulty assigning support requests
- Scattered support communication

SupportHub Solution:

SupportHub provides a centralized workspace where administrators can manage users, assign roles, allocate tickets, and monitor the complete support workflow.



04. ARCHITECTURE & TECHNOLOGY

SupportHub follows an Android application architecture based on **MVVM and Firebase**.

Language: Kotlin

UI: Jetpack Compose

Architecture: MVVM

Design System: Material 3

Backend: Firebase

Authentication: Firebase Authentication

Database: Cloud Firestore

State Management: StateFlow

Async Operations: Kotlin Coroutines

Navigation: Navigation Compose

Pattern: Repository Pattern

Authentication Provider: Google Sign-In + Email/Password

Architecture

User → Android App → ViewModel → Repository → Firebase Authentication / Firestore

The documented architecture separates the Android frontend from Firebase-powered backend services and uses role-based dashboards for Employees, Agents, and Administrators.

05. CORE SUPPORT WORKFLOW

The main SupportHub workflow connects authentication, ticket creation, administration, assignment, and resolution.

Login → Workspace → Create Ticket → Firebase → Admin Review → Agent Assignment → Ticket Resolution → Employee Update

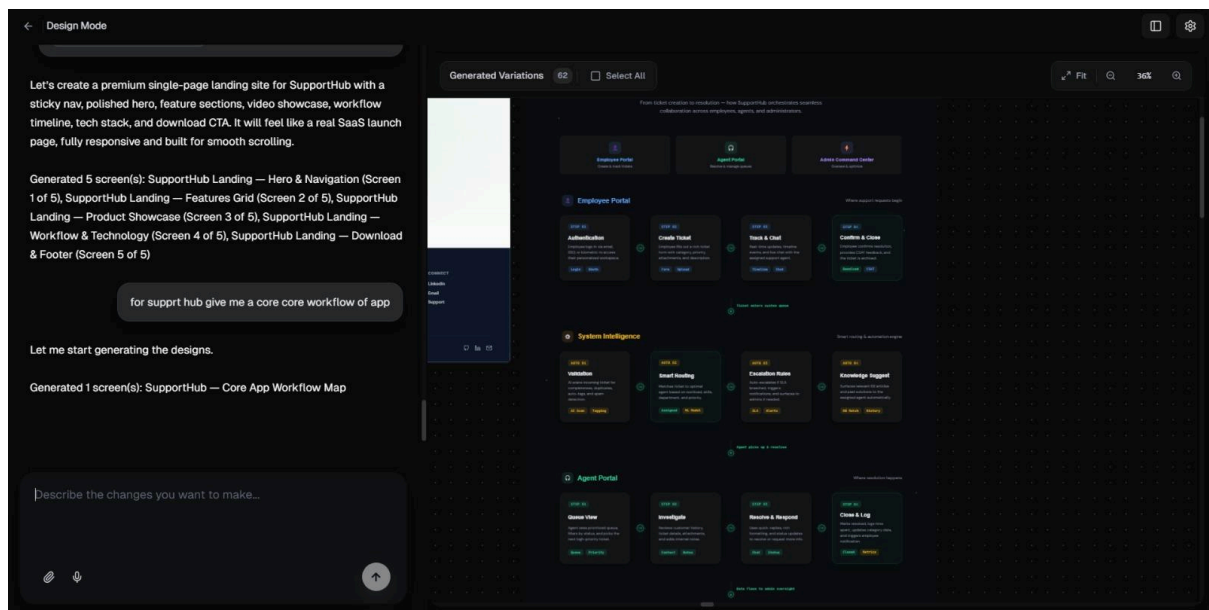
Workflow

Employees create support tickets and submit them to the system.

Administrators review users and tickets, approve users, assign roles, and allocate tickets to available agents.

Agents work on assigned tickets and update the status from **Open** → **In Progress** → **Resolved**.

Employees receive the latest ticket status through real-time Firebase synchronization.



06. FEATURE MAP

Authentication

Email/Password and Google Sign-In authentication with role-based access.

Employee Panel

Create tickets, view tickets, track status, search tickets, and manage profile.

Agent Panel

View assigned tickets, update ticket status, resolve issues, and manage support work.

Admin Panel

Approve users, assign roles, allocate tickets, manage users, and monitor support activity.

Ticket Management

Create, assign, track, update, and resolve support tickets.

Search

Search and filter tickets, employees, and agents.

Notifications

Receive updates related to ticket creation, assignment, status changes, and resolution.

Profile & Settings

Manage profile information, application preferences, and account settings.

Workspace

Manage organization-specific support operations.

The documented application includes dedicated dashboards and role-specific workflows for all three user types.

07. BUILD: DESIGN MODE

Objective

Create a premium visual system for SupportHub covering the authentication flow and the three primary application portals.

Design Direction

- Premium Android UI
- Material 3
- Light enterprise theme
- Emerald and blue accents
- Modern typography
- Rounded cards
- Responsive layouts
- Premium phone mockups
- Consistent design system

Design Mode was used to generate separate visual experiences for:

Authentication

Splash, Login, Register, Forgot Password and Workspace Loading.

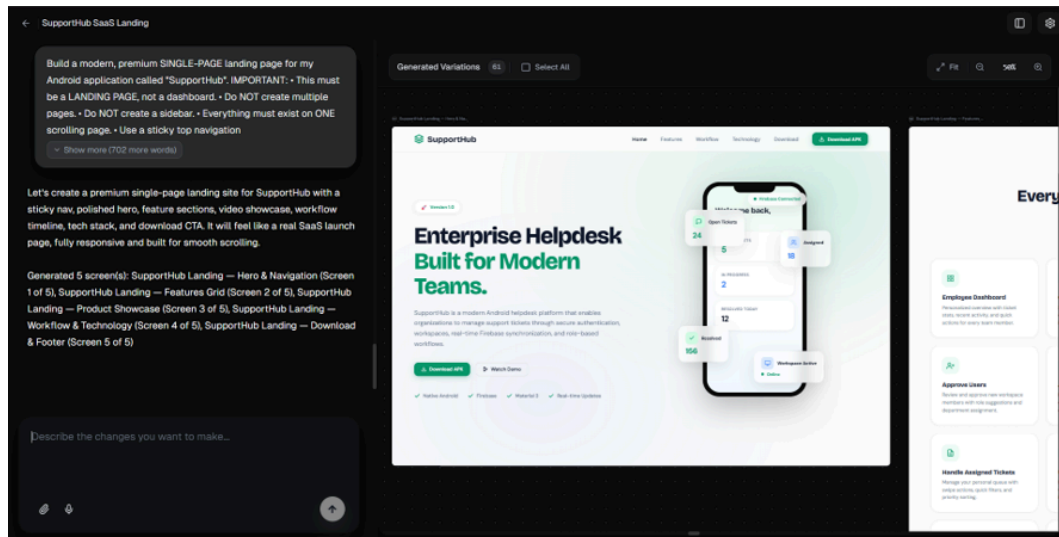
Employee Portal

Dashboard, Create Ticket, My Tickets, Ticket Details, Chat, Notifications, Knowledge Base, Search, Profile and Settings.

Agent Portal

Dashboard, Ticket Queue, Ticket Details, Customer Details, Chat, Search, Notifications, Performance, Profile and Settings.

The Design Mode prompt generated more than 50 connected screens using shared color, typography, spacing and component styles.



Prompts Used

Prompt 1 : App Launch & Authentication

Design a premium Behance-style mobile UI showcase for the App Launch and Authentication flow of "SupportHub", a modern enterprise helpdesk platform. Display five realistic Android phone mockups in a clean editorial layout: Splash Screen, Login, Register, Forgot Password, and Workspace Loading. Use premium rounded cards, professional CTA buttons, subtle enterprise illustrations, elegant motion, and a minimal light theme. Use Material 3 Expressive, Apple Human Interface, Linear, Notion, Arc Browser, and Google Pixel UI as inspiration.

Prompt 2 : Employee Portal

Design a premium Behance-style showcase board for the Employee Portal of "SupportHub". Display high-fidelity Android mockups for Employee Dashboard, Create Ticket, My Tickets, Ticket Details, Real-time Chat, Notifications, Knowledge Base, FAQ, Search, Profile, and Settings. The dashboard should include a personalized greeting, Create Ticket hero card, ticket statistics, recent activity, announcements, and quick actions. Ticket Details should include a timeline, status chips, attachments, assigned agent information, and activity history.

Prompt 3 : Agent Portal

Design a premium Android UI case study board for the Agent Portal of "SupportHub". Display realistic phone mockups for Agent Dashboard, Ticket Queue, Ticket Details, Customer Details, Chat, Search & Filters, Notifications, Performance Dashboard, Profile, and Settings. The Agent Dashboard should feel like a modern productivity workspace with KPI cards, today's queue, assigned tickets, response time, and quick actions. The Queue should include premium filters, priority chips, swipe actions, customer avatars, and elegant ticket cards.

Prompt 4 : Admin Portal

Design a premium Android UI case study board for the Administrator Portal of "SupportHub". Create a complete admin experience for managing users, roles, workspaces, tickets and support operations. Use a professional Material 3 enterprise interface with dashboards, management cards, ticket allocation, user approval, role management, analytics and settings. Maintain a consistent visual language with the Employee and Agent portals while making the Admin experience clearly focused on system-level control.

08. BUILD: PROTOTYPE MODE

Objective

Convert the approved SupportHub landing-page designs into a functional and responsive website.

Implementation

Build was used to create the SupportHub product landing page with:

- Hero section
- Product showcase
- Feature section
- Workflow section
- Technology section
- Download section
- Demo video previews
- Responsive layouts
- Smooth animations
- Navigation
- APK download integration

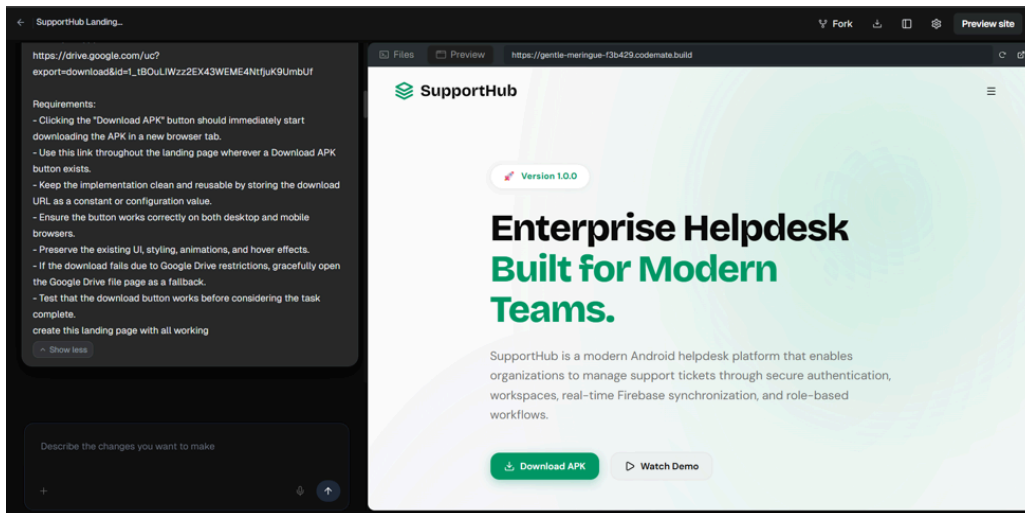
The approved designs were converted into a production-ready landing page without redesigning the visual direction.

Demo Integration

The landing page includes four product demo videos:

- Employee Dashboard
- Ticket Creation
- Admin Dashboard
- Agent Dashboard

Each video is presented as a product showcase inside an Android phone mockup.



Prompts Used

Prompt 1 : SupportHub Landing Page

Build a complete SupportHub landing page using the approved designs already created. Convert the designs into a fully responsive, production-ready website with smooth animations and a premium UI. Do not redesign anything. Use the existing designs exactly and make every section responsive across desktop, tablet and mobile.

Prompt 2 : Product Workflow

Create a beautiful animated SupportHub product workflow showing the complete support process: User Login → Workspace → Create Ticket → Saved to Firebase → Admin Reviews → Agent Assigned → Agent Resolves Ticket → Employee Receives Update. Present each step clearly with smooth scrolling animations and a professional enterprise helpdesk visual style.

Prompt 3 : Technology & Download Experience

Add a dedicated technology section to the SupportHub landing page showcasing Kotlin, Jetpack Compose, MVVM, Firebase Authentication, Cloud Firestore, StateFlow, Coroutines, Navigation Compose, Material 3, Repository Pattern and Google Sign-In. Follow this with a premium download section containing the SupportHub version, Android platform information, APK download button, demo video and relevant project links.

Prompt 4 : Final Functional Build

Make the complete SupportHub landing page fully functional and responsive. Add smooth scrolling, page transitions, hover effects, loading animations, configurable APK download functionality and embedded demo videos. Use clean reusable components and production-ready code. Ensure every button, animation, navigation element and video preview works correctly without broken links or placeholder functionality.

09. BUILD PROMPT HIGHLIGHTS

Design Mode

Design a premium Behance-style mobile UI showcase for the App Launch and Authentication flow of "SupportHub", a modern enterprise helpdesk platform.

The prompt defined the authentication screens, Employee Portal, Agent Portal, visual system, phone mockups, typography, colors, spacing, and connected design system.

Prototype Mode

Build the complete SupportHub Landing Page using all the approved designs already created.

The prompt instructed Build to convert the approved designs into a responsive website with animations, demo videos, APK download functionality, reusable components, and working navigation.

Prompts Used

Prompt 1 : Product Experience

Create a premium product experience for SupportHub, a modern enterprise helpdesk platform. Showcase the application through polished Android UI designs covering authentication, employee workflows, support agent operations and administrator controls. Maintain a consistent Material 3 visual system while giving each role a clear and purpose-driven experience.

Prompt 2 : Employee Experience

Create a complete Employee experience for SupportHub where users can access their dashboard, create support tickets, view existing tickets, track ticket status, communicate through chat, access the knowledge base and manage their profile and settings. Keep the interface intuitive, responsive and focused on making support requests simple to create and track.

Prompt 3 : Agent Experience

Create a complete Support Agent experience focused on managing assigned tickets efficiently. Include the agent dashboard, ticket queue, ticket details, customer information, chat, search and filters, notifications, performance information and profile settings. Use real-time ticket information and a premium productivity-focused Material 3 interface.

Prompt 4 : Landing Page

Build a professional SupportHub product landing page that introduces the application, explains the support workflow, presents the underlying Android technologies, showcases the product through demo videos and provides a clear APK download experience. Keep the entire experience within a single scrolling page with smooth navigation and premium animations.

10. CORA: PLANNING & IMPLEMENTATION

CORA was used throughout the Android application development process for architecture planning, implementation, debugging, UI refinement, Firebase integration, navigation fixes, and feature development.

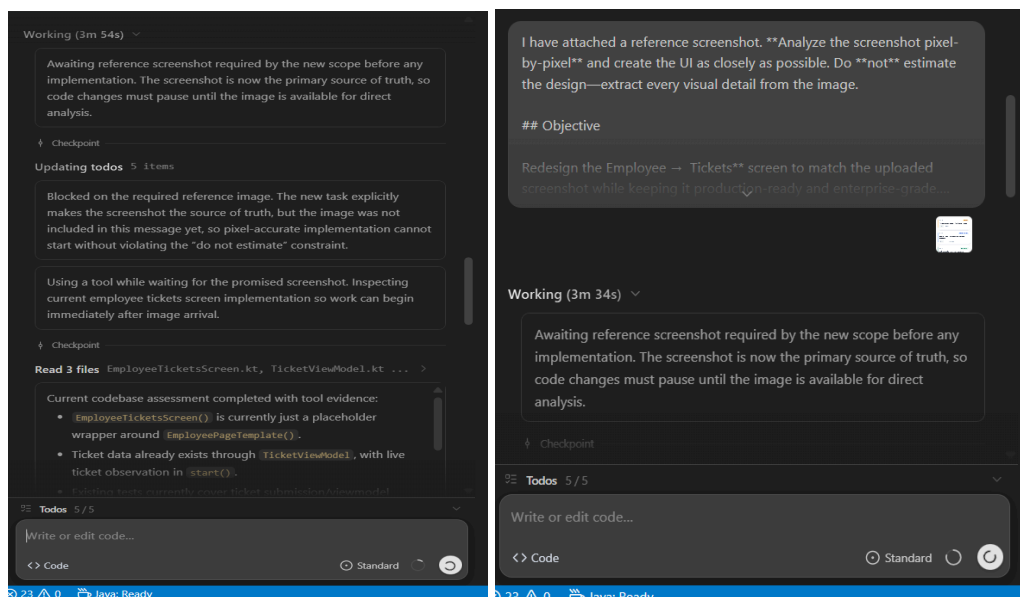
Main Objectives

- Plan the SupportHub architecture
- Establish the Firebase structure
- Implement authentication
- Build role-based dashboards
- Implement ticket management
- Connect Firestore data
- Fix navigation flows
- Improve UI
- Verify real-time synchronization

Planning Flow

Project Planning → Architecture → Design → Implementation → Firebase Integration → Testing → Debugging → Refinement

The initial planning prompts focused on understanding ITSM projects, planning a Firebase-based ticket management application, defining the architecture, and establishing



CORA Prompt Highlights

Prompt 1 : Firebase & Authentication Integration

Connect the existing SupportHub Android application with the existing Firebase project using the `google-services.json` file already present inside the app module. Configure and verify Firebase Authentication, Email/Password authentication, Google Sign-In, Cloud Firestore, and the required Gradle plugins and dependencies. Ensure Firebase initializes correctly, authentication persists across app restarts, Firestore read/write operations work without permission issues, and real-time synchronization functions correctly. Preserve the existing MVVM architecture, Repository Pattern, ViewModels, Navigation Graph, package structure, Firestore collections, document schema, and business logic. Do not create a new Firebase project, rename collections, modify the existing schema, or unnecessarily rewrite repositories and authentication logic. Fix only the configuration or integration issues required to make the existing Firebase setup fully functional and production-ready.

Prompt 2 : Complete Authentication & Role-Based Flow

Review and complete the entire SupportHub authentication and onboarding flow. After successful registration or Google Sign-In, create or retrieve the user's Firestore document using the Firebase Authentication UID and preserve fields such as `requestedRole`, `approvedRole`, `status`, `workspaceName`, and authentication information. Navigate users through Complete Profile and Workspace Loading when required, then check the latest Firestore status and `approvedRole` before deciding where they should go. Users with pending status must be shown the Pending Approval screen, while active users must be redirected to the correct Employee, Agent, or Admin dashboard based only on `approvedRole`. Do not hardcode roles, create duplicate user documents, change the Firestore schema, or break the existing MVVM architecture and navigation graph. Prevent duplicate navigation events, white screens, skipped screens, flickering, and unauthorized dashboard access.

Prompt 3 : Complete Role-Based Dashboards

Build the complete role-based dashboard system for the SupportHub Android application with three independent experiences: Employee, Support Agent, and Administrator. Each role must have its own dashboard, navigation flow, features, Firebase queries, screens, bottom navigation, and user experience while maintaining a consistent premium Material 3 enterprise design. All dashboard statistics, tickets, users, approvals, assignments, and activities must come directly from Cloud Firestore using real-time snapshot listeners or Kotlin Flow. Do not use mock, dummy, or hardcoded data. Employees should manage their support requests, agents should manage assigned tickets and performance, and administrators should manage users, approvals, roles, ticket assignments, and analytics. Enforce access using the Firestore `approvedRole` field and ensure

users can never access another role's dashboard or features. Preserve the existing MVVM architecture, repositories, ViewModels, Firestore schema, authentication logic, and business rules.

Prompt 4 Employee Ticket Management & Real-Time Firebase

Build and refine the complete Employee Ticket Management module for SupportHub using Kotlin, Jetpack Compose, Material 3, MVVM Architecture, and Firebase Cloud Firestore. Allow employees to create tickets with title, description, category, priority, and other required information, then save them to Firestore with the logged-in employee's Firebase UID and the appropriate default status. Provide My Tickets, Ticket Details, search, filtering, sorting, ticket cards, ticket timeline, and real-time status tracking. Only show tickets belonging to the currently authenticated employee and use real-time Firestore listeners so newly created tickets, assignments, status changes, priority changes, resolution notes, and closures appear automatically without manual refresh. Implement loading, empty, offline, and error states while optimizing queries and preventing duplicate listeners. Preserve the existing repositories, ViewModels, navigation graph, Firestore schema, authentication flow, and business logic, and enhance the existing implementation instead of creating unnecessary duplicate files.

11. CORA TASK ARCHIVE

Authentication

CORA was used to implement and refine:

- Email/Password authentication
- Google Sign-In
- Session persistence
- Logout
- Forgot Password
- Role-based authentication
- Approval status handling

Firestore

- User data
- Tickets
- Notifications
- Workspaces
- Real-time listeners
- CRUD operations
- Repository integration

Ticket Management

- Ticket creation
- Ticket assignment
- Status updates
- Priority handling
- Search and filtering
- Real-time synchronization

Dashboard

- Live ticket statistics
- Recent tickets
- Assigned tickets
- Notifications
- Role-specific dashboard data

Profile & Settings

- Profile management
- Password handling
- Profile images
- Theme settings

- Help and support

These implementation tasks are documented across the SupportHub prompt sheet, including authentication, Firestore, ticket management, dashboards, profile/settings, search, and validation.

i want my
C:\Users\swsdh\AndroidStudioProjects\SupportHub\app\src\main\java\com\example\supporthub\features\authentication\screens\PendingApprovalScreen.kt this screen to be same theme as forgot screen but only the ui keep all things as like that only chnge the ui like white background and all which will be same as other authentication screen

Worked for 4m 58s >

Updated only the UI in `PendingApprovalScreen.kt` so it now matches the same white-background authentication theme used by the other auth screens.

What changed:

- Replaced the plain scaffold/top-bar layout with the same soft white authentication-style background and blurred color accents.
- Added a centered rounded card matching the visual style of `ForgotPasswordScreen.kt`.
- Kept the same pending-approval content, workspace details, status information, refresh action, and logout action.
- Changed only the presentation layer; existing `onRefresh`, `onLogout`, and displayed `User` data remain intact.

Verification evidence:

- Ran `gradlew.bat :app:compileDebugTest`
- Result: **BUILD SUCCESSFUL**

Scope control:

- Modified only `PendingApprovalScreen.kt`:

Root cause fixed in the startup flow.

Findings:

- `[SplashScreen()]`
(`app/src/main/java/com/example/supporthub/features/splash/SplashScreen.kt`) previously performed its own Firebase Auth + Firestore lookup and navigated immediately.
- `[AppNavGraph()]`
(`app/src/main/java/com/example/supporthub/core/navigation/AppNavGraph.kt`) then tried to render `[workspaceLoadingScreen()]`
(`app/src/main/java/com/example/supporthub/features/authentication/screens/workspaceLoadingScreen.kt`) only when `[uiState.user]`
(`app/src/main/java/com/example/supporthub/features/authentication/utils/AuthUiState.kt`) was already populated, otherwise it returned from the route body. That created the blank white screen on cold reopen: splash navigated before viewmodel state had finished hydrating.
- Startup routing was split across UI layers, which created race conditions and duplicate-navigation risk.

What changed:

- Added centralized startup destination resolution in `[StartupDestinationResolver]`
(`app/src/main/java/com/example/supporthub/features/authentication/utils/StartupDestinationResolver.kt`).
- Added startup state to `[AuthUiState]`
(`app/src/main/java/com/example/supporthub/features/authentication/utils/Auth`

12. IMPLEMENTATION MILESTONES

01. Planning

Defined the ITSM concept, architecture, Firebase structure, and three user roles.

02. Design

Created the authentication, Employee, Agent, and Admin interface designs using Build Design Mode.

03. Application Development

Implemented the Android application using Kotlin, Jetpack Compose, MVVM, and Material 3.

04. Firebase Integration

Connected Authentication, Firestore, real-time listeners, and role-based user data.

05. Role-Based Workflow

Implemented separate Employee, Agent, and Admin experiences.

06. Ticket Management

Implemented ticket creation, assignment, tracking, status updates, and resolution.

07. Landing Page

Converted the approved Build designs into a responsive SupportHub landing page.

08. Testing & Refinement

Used CORA to investigate issues, refine implementation, and verify application behavior.

13. BACKEND, DATA & INTEGRATIONS

SupportHub uses Firebase as the backend platform.

Firestore Services

Firestore Authentication

Handles Email/Password and Google Sign-In.

Firestore Cloud Firestore

Stores and synchronizes users, workspaces, tickets, notifications, and related application data.

Firestore Storage

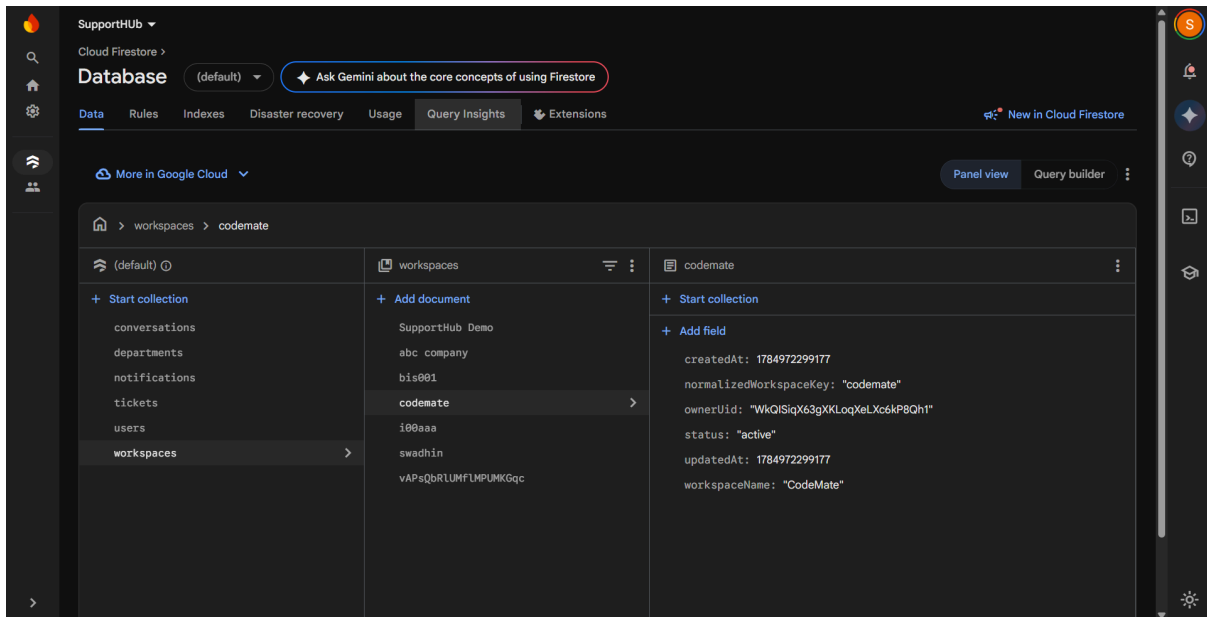
Supports application storage requirements where configured.

Core Data

- Users
- Workspaces
- Tickets
- Notifications
- Employee information
- Agent information
- Role and approval data

The documented database structure includes **Users**, **Workspaces**, and **Tickets** as core Firestore collections.

(default) + Start collection departments messages notifications tickets users workspaces	users + Add document 0An4JfbBofxjEkLuf0Re BwPQKgewJgbKuQWK0nqLckJkuzx2 V8bL1jL96KP5ninpi4YCRKjW0Bd2	BwPQKgewJgbKuQWK0nqLckJkuzx2 + Start collection + Add field approvedRole: "admin" authProvider: "google" createdAt: 1784279518274 email: "swsdhindwibedy@gmail.com" fullName: "Swadhin Dwibedy" lastLogin: 1784293976747 requestedRole: "admin" status: "active" uid: "BwPQKgewJgbKuQWK0nqLckJkuzx2" updatedAt: 1784279518274 workspaceName: "swadhin" workspaceOwner: true
---	--	---



14. AUTHENTICATION & USER MANAGEMENT

SupportHub uses **Firebase Authentication** with Email/Password and Google Sign-In.

After authentication, the application retrieves the user's Firestore profile and checks the user's approval status and assigned role.

User Flow

Login → Firebase Authentication → Firestore User Profile → Approval Status → Approved Role → Role Dashboard

Roles

Employee

Accesses the Employee dashboard and ticket management features.

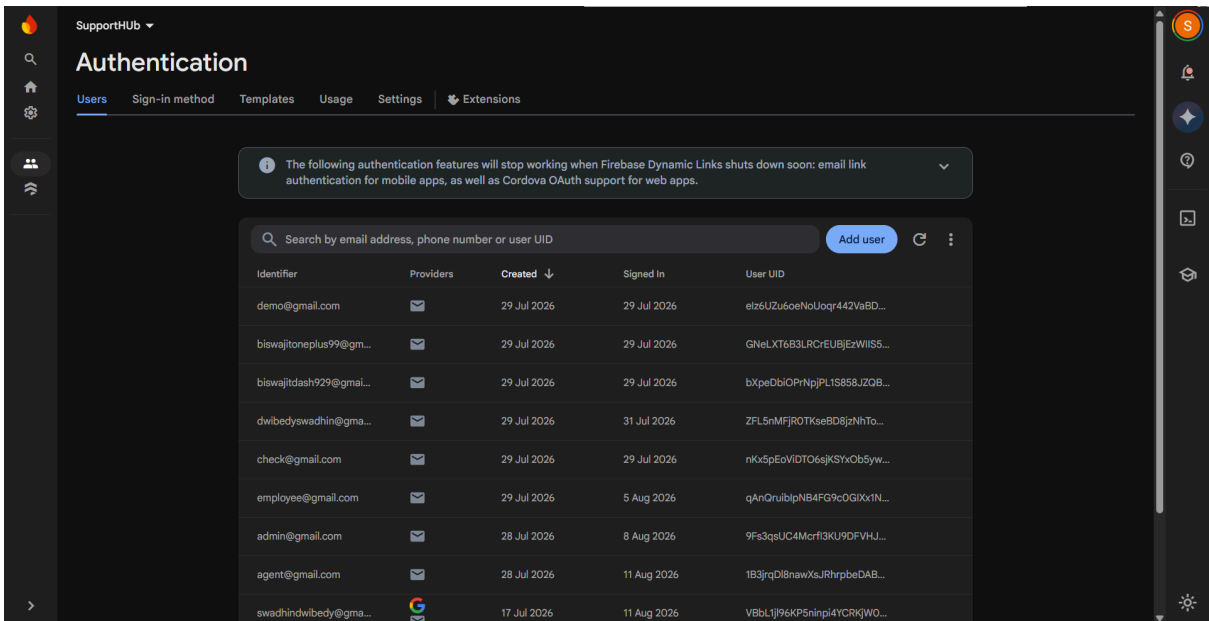
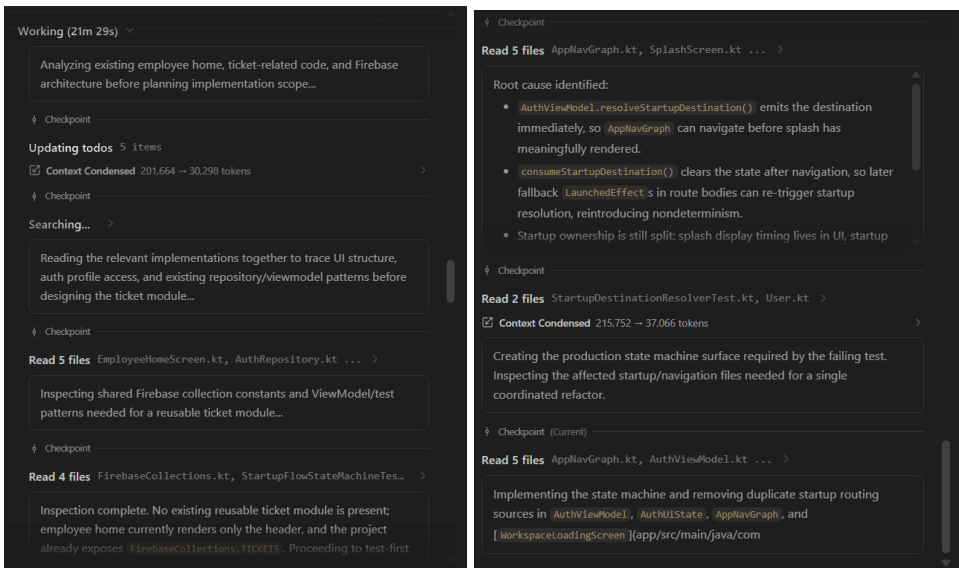
Agent

Accesses assigned support tickets and resolution workflows.

Admin

Approves users, assigns roles, and manages tickets and support operations.

The system uses role-based access and redirects authenticated users to the appropriate dashboard after validation.



15. RESPONSIVE UI & POLISH

SupportHub follows a modern Material 3 design approach focused on usability and consistency.

UI Characteristics

- Material 3 components
- Responsive layouts
- Consistent spacing
- Rounded cards
- Modern typography
- Smooth animations
- Loading states
- Empty states
- Error states
- Light and dark theme support

The Build landing page also follows a responsive SaaS design with mobile support, animations, modern typography, rounded surfaces, and a white and emerald visual theme.

